



EA.3 - PROGRAMACIÓN FUNCIONAL.

Tecnología en desarrollo de software.

ASIGNATURA:

Programación II: Orientada a objetos avanzada.

INTEGRANTES:

Valentina Stephany Kassar Acuña.

Lina Marcela Muñoz.

Maria Ines Julio Navarro.

(Grupo Actividad 3 5)

DOCENTE:

Boris Alberto Salleg Royero.

Septiembre 27 2026

ÍNDICE.

1. Introducción.....	3
2. Objetivos.....	4
2.1 Objetivo general.....	4
2.2 Objetivos específicos.....	4
3. Desarrollo.....	5
3.1 Diseño de caso de estudio.....	5
3.2 Explicación funciones puras.....	5
3.3 Explicación inmutabilidad.....	5
3.4 Explicación de funciones de orden superior.....	5
3. 5 Explicación lambdas.....	6
3.6 Explicación Streams.....	6
4. Evidencia de ejecución (Capturas de pantalla.).....	7
5. Links.....	8
5.1 Link video.....	8
5.2 Link repositorio.....	8
6. Conclusión.....	9

INTRODUCCIÓN.

Como grupo, abordamos esta actividad con el propósito de aplicar los principios de la programación funcional en Java, construyendo una solución declarativa que permita manejar grandes volúmenes de datos de manera eficiente. La programación funcional se caracteriza por el uso de funciones puras, la inmutabilidad de los datos y la composición de operaciones de alto orden, lo cual garantiza claridad, mantenibilidad y ausencia de efectos secundarios en el código.

Para este caso de estudio, modelamos un sistema de gestión de biblioteca, transformando las estructuras tradicionales de un modelo de dominio (Libros, Autores, Personas, Préstamos y Libros Digitales) hacia un paradigma declarativo apoyado por Streams y Java Records.

El trabajo colaborativo se realizó mediante GitHub, donde cada integrante aportó al repositorio principal y mantuvo sincronización de cambios, asegurando transparencia y trazabilidad en el desarrollo y control de versiones.

OBJETIVOS.

2.1 Objetivo General.

Implementar un caso de estudio de gestión de biblioteca en Java aplicando el paradigma funcional, documentando el proceso técnico y alojando la solución en un repositorio GitHub bajo normas de entrega académica..

2.2 Objetivos Específicos.

- Aplicar funciones puras para garantizar resultados deterministas sin efectos secundarios en el sistema.
- Implementar inmutabilidad en las estructuras de datos utilizando Java Records y colecciones no modificables.
- Utilizar expresiones lambdas y funciones de orden superior para simplificar operaciones sobre colecciones.
- Emplear streams para el procesamiento declarativo de grandes cantidades de datos.
- Mantener un código limpio, modular, legible y alineado con las convenciones estándar de Java.
- Evidenciar el trabajo colaborativo mediante repositorios individuales y grupales en GitHub.

Desarrollo

3.1 Diseño del caso de estudio.

El sistema modelado corresponde a una **gestión de biblioteca funcional**, inspirado en el trabajo previo de diagramas UML. Se implementaron las entidades principales y más destacadas:

- **Libro**: con atributos como título, autor y disponibilidad.
- **Usuario**: con información de cuenta y préstamos asociados.
- **Préstamo**: con fechas y estado (activo, devuelto, En_Restraso).
- **LibroDigital**: extensión de Libro con atributos de descarga.

3.2 Explicación funciones puras.

- **Funciones puras**: Las funciones puras son el pilar fundamental del paradigma funcional. Una función se considera pura si satisface dos condiciones:
 - **Determinismo**. Siempre devuelve el mismo resultado ante los mismos argumentos de entrada.
 - **Ausencia de efectos secundarios**. No modifica el estado externo del programa (Variables globales, colecciones fuera de su ámbito) ni realiza acciones mutables.
 - **Aplicación en el código**. Métodos analíticos como PrestamosPorCategorias() o UsuariosConAtrasos() reciben una lista de préstamos y producen un nuevo resultado (Map o List) sin alterar la lista original ni modificar ningún atributo de las entidades.

3.3 Explicación inmutabilidad.

La inmutabilidad establece que una vez que un objeto es creado, su estado no puede ser alterado. Esto evita condiciones de carrera en sus entornos concurrentes y garantiza la consistencia del sistema.

Aplicación en el código. Se aplicaron las entidades del modelo (Libro, Autor, Persona, Préstamo, LibroDigital y Biblioteca) mediante la característica Record. Los Records definen campos final inmutables por defecto. Asimismo, las colecciones fuente se tratan con [List.of\(\)](#) o interacciones inalterables.

3.4 Explicación de funciones de orden superior.

Una función de orden superior es aquella que puede recibir una o más funciones como parámetros, o bien devolver una función como resultado.

Aplicación en el código. Los métodos intermedios de la Stream (`.filter()`, `.map()`, `.sorted()`, `.collect()`). Son funciones de orden superior. Por ejemplo, en el método `detectarAltaCategorias()`, el método `.collect()` recibe una función definitoria de agrupación y filtrado mediante un `Predicate<Long>`.

3.5 Explicación lambdas.

Las expresiones Lambdas son funciones anónimas que permiten definir comportamientos al vuelo de forma concisa y expresiva.

Aplicación en el código.

- Filtrado de préstamos morosos: $p \rightarrow p.estado() == EstadoPrestamo.EN_ATRASSO$
- Agrupación por categoría: $p \rightarrow p.libro().categoria()$
- En conjunto con referencias a métodos (*method references*) como `Prestamo::usuario` o `Prestamo::fechaPrestamo`.

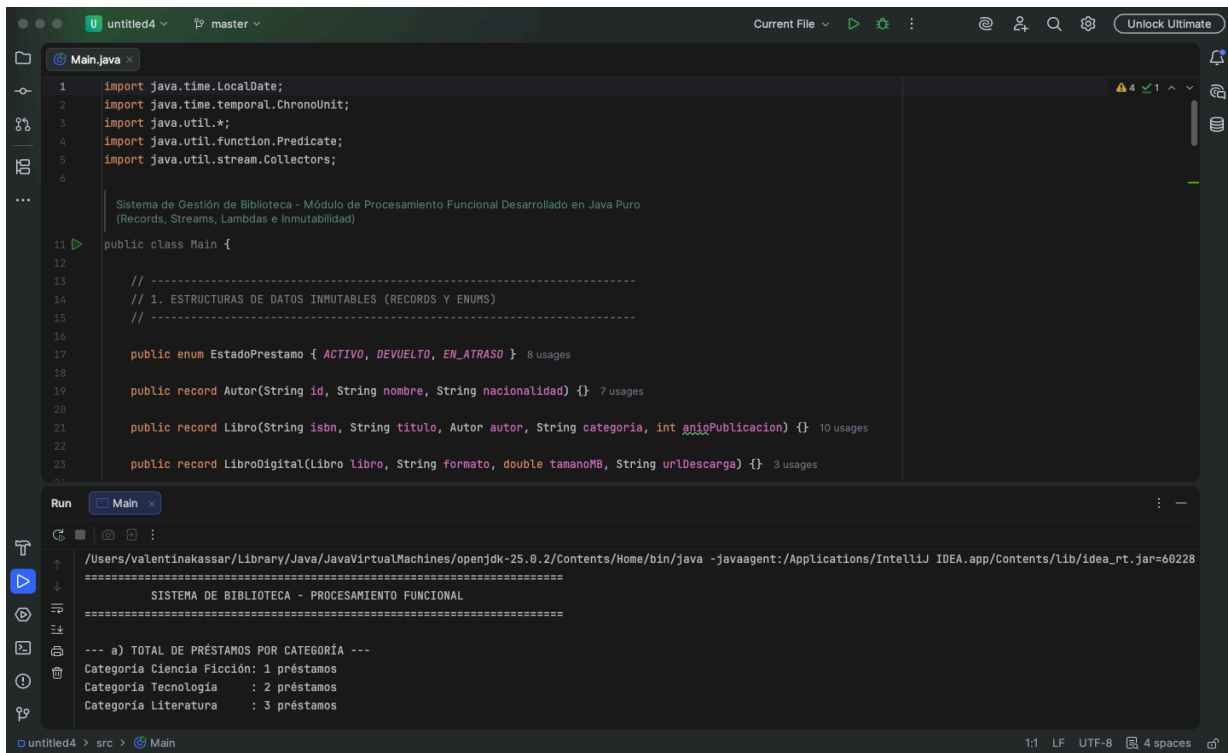
3.6 Explicación Stream.

La Stream de Java permite procesar datos de forma declarativa mediante una canalización de operaciones que transforman datos sin modificar la fuente original.

Aplicación en el código.

- **Operaciones intermedias:** `.filter()`, `.map()`, `.sorted()`, `flatMap()`.
- Operaciones terminales: `.toList()`, `.collect(Collectors.groupingBy())`, `.collect(Collectors.toMap())`.

EVIDENCIA DE EJECUCIÓN. (CAPTURAS DE PANTALLA).

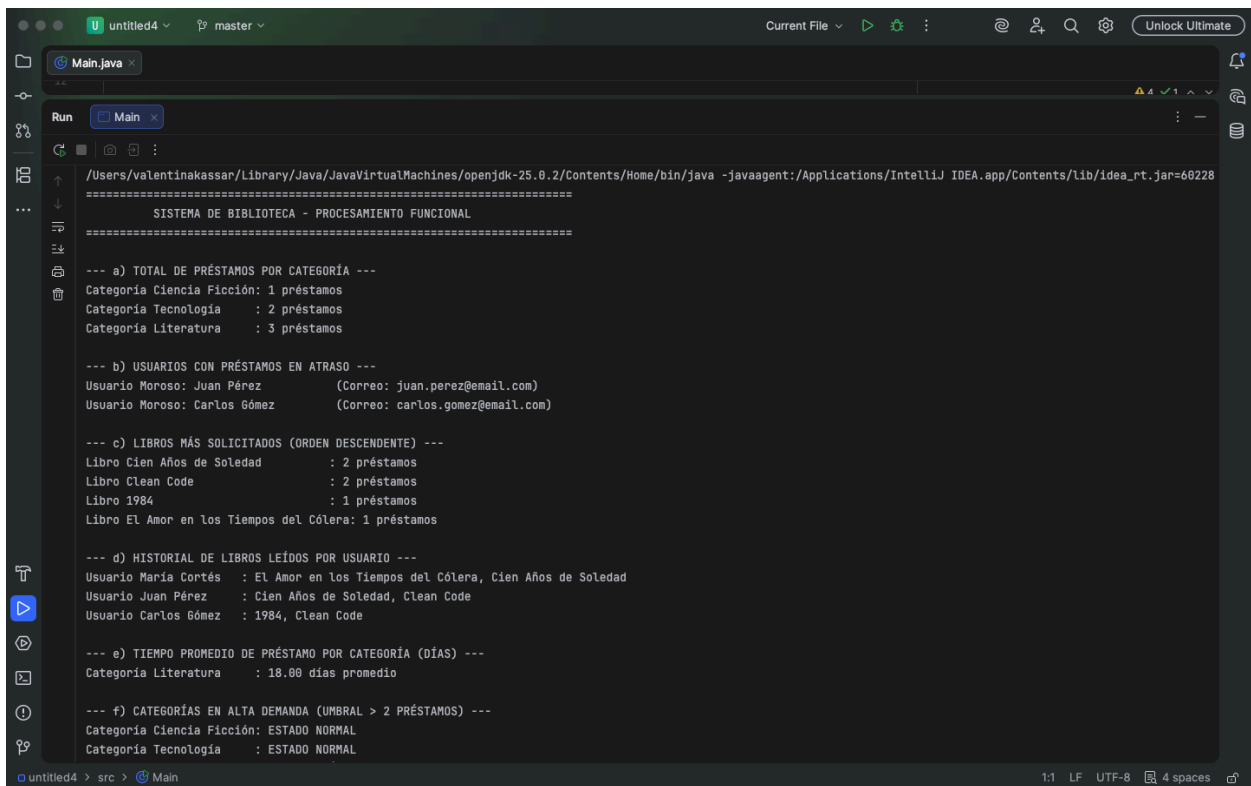


The screenshot shows the IntelliJ IDEA IDE with the file `Main.java` open. The code defines a system for managing a library, including imports for date, time, and stream utilities, and a `Main` class with several records and an enum.

```
1 import java.time.LocalDate;
2 import java.time.temporal.ChronoUnit;
3 import java.util.*;
4 import java.util.function.Predicate;
5 import java.util.stream.Collectors;
6
7
8 Sistema de Gestión de Biblioteca - Módulo de Procesamiento Funcional Desarrollado en Java Puro
9 (Records, Streams, Lambdas e Inmutabilidad)
10
11 public class Main {
12
13     // -----
14     // 1. ESTRUCTURAS DE DATOS INMUTABLES (RECORDS Y ENUMS)
15     // -----
16
17     public enum EstadoPrestamo { ACTIVO, DEVUELTO, EN_ATRASO } 8 usages
18
19     public record Autor(String id, String nombre, String nacionalidad) {} 7 usages
20
21     public record Libro(String isbn, String titulo, Autor autor, String categoria, int anioPublicacion) {} 10 usages
22
23     public record LibroDigital(Libro libro, String formato, double tamañoMB, String urlDescarga) {} 3 usages
24 }
```

The Run window shows the execution output:

```
/Users/valentinakassar/Library/Java/JavaVirtualMachines/openjdk-25.0.2/Contents/Home/bin/java -javaagent:/Applications/IntelliJ IDEA.app/Contents/Lib/idea_rt.jar-60228
=====
SISTEMA DE BIBLIOTECA - PROCESAMIENTO FUNCIONAL
=====
--- a) TOTAL DE PRÉSTAMOS POR CATEGORÍA ---
Categoría Ciencia Ficción: 1 préstamos
Categoría Tecnología      : 2 préstamos
Categoría Literatura      : 3 préstamos
```



The screenshot shows the IntelliJ IDEA IDE with the file `Main.java` open. The code defines a system for managing a library, including imports for date, time, and stream utilities, and a `Main` class with several records and an enum.

```
1 import java.time.LocalDate;
2 import java.time.temporal.ChronoUnit;
3 import java.util.*;
4 import java.util.function.Predicate;
5 import java.util.stream.Collectors;
6
7
8 Sistema de Gestión de Biblioteca - Módulo de Procesamiento Funcional Desarrollado en Java Puro
9 (Records, Streams, Lambdas e Inmutabilidad)
10
11 public class Main {
12
13     // -----
14     // 1. ESTRUCTURAS DE DATOS INMUTABLES (RECORDS Y ENUMS)
15     // -----
16
17     public enum EstadoPrestamo { ACTIVO, DEVUELTO, EN_ATRASO } 8 usages
18
19     public record Autor(String id, String nombre, String nacionalidad) {} 7 usages
20
21     public record Libro(String isbn, String titulo, Autor autor, String categoria, int anioPublicacion) {} 10 usages
22
23     public record LibroDigital(Libro libro, String formato, double tamañoMB, String urlDescarga) {} 3 usages
24 }
```

The Run window shows the execution output:

```
/Users/valentinakassar/Library/Java/JavaVirtualMachines/openjdk-25.0.2/Contents/Home/bin/java -javaagent:/Applications/IntelliJ IDEA.app/Contents/Lib/idea_rt.jar-60228
=====
SISTEMA DE BIBLIOTECA - PROCESAMIENTO FUNCIONAL
=====
--- a) TOTAL DE PRÉSTAMOS POR CATEGORÍA ---
Categoría Ciencia Ficción: 1 préstamos
Categoría Tecnología      : 2 préstamos
Categoría Literatura      : 3 préstamos

--- b) USUARIOS CON PRÉSTAMOS EN ATRASO ---
Usuario Moroso: Juan Pérez      (Correo: juan.perez@email.com)
Usuario Moroso: Carlos Gómez    (Correo: carlos.gomez@email.com)

--- c) LIBROS MÁS SOLICITADOS (ORDEN DESCENDENTE) ---
Libro Cien Años de Soledad      : 2 préstamos
Libro Clean Code                : 2 préstamos
Libro 1984                      : 1 préstamos
Libro El Amor en los Tiempos del Cólera: 1 préstamos

--- d) HISTORIAL DE LIBROS LEÍDOS POR USUARIO ---
Usuario María Cortés           : El Amor en los Tiempos del Cólera, Cien Años de Soledad
Usuario Juan Pérez              : Cien Años de Soledad, Clean Code
Usuario Carlos Gómez            : 1984, Clean Code

--- e) TIEMPO PROMEDIO DE PRÉSTAMO POR CATEGORÍA (DÍAS) ---
Categoría Literatura            : 18.00 días promedio

--- f) CATEGORÍAS EN ALTA DEMANDA (UMBRAL > 2 PRÉSTAMOS) ---
Categoría Ciencia Ficción: ESTADO NORMAL
Categoría Tecnología      : ESTADO NORMAL
```

LINKS.

5.1 Link video.

5.2 Link repositorio.

CONCLUSIONES.

El trabajo permitió comprender cómo la programación funcional aporta claridad y robustez al desarrollo de software. Se evidenció que el uso de funciones puras y estructuras inmutables reduce errores y facilita el mantenimiento. Además, el enfoque declarativo mediante streams simplifica el manejo de grandes volúmenes de datos.

A nivel profesional, esta experiencia fortalece la capacidad de diseñar soluciones escalables y colaborativas, integrando buenas prácticas de programación y control de versiones.